A Project Proposal Report on

Survival Days: A Zombie Survival Simulator

A Project Report Submitted for the Course

Object-Oriented Programming (OOP)

Bachelor of Science In Computer Science

Submitted by:

Simran Silpakar, WU0257727

Date:

08/14/2026

King's College Nepal

Bijulibajar, Kathmandu

ABSTRACT

Survival Days is a console-based, turn-based survival game developed in C++ to demonstrate core Object-Oriented Programming (OOP) principles through an interactive, playable system rather than isolated exercises. The player manages four interdependent resources, health, hunger, food, and bullets, across a series of in-game days, choosing each turn to search the surroundings, fight a zombie, eat food, or quit. Random events drive replayability: searching may yield food, ammunition, nothing, or an ambush by a zombie, while every zombie is spawned with a randomized attack strength.

The system is built around three cooperating classes, `Player`, `Zombie`, and `Game`, that together illustrate encapsulation (private state accessed only through public methods), composition (the `Game` class owns a `Player` and creates `Zombie` objects as encounters occur), and constructor-based initialization (each class establishes its own starting state). The game loop continues until the player's health or hunger is depleted, at which point the total number of days survived is reported as the player's final score. This proposal presents the project's objectives, scope, design, and planned deliverables, including UML class, use case, and sequence diagrams describing the system.

Keywords: Zombie Survival, C++, Object-Oriented Programming, Console Application, Resource Management, UML Design.

TABLE OF CONTENTS

LIST OF FIGURES

1. INTRODUCTION

This project applies Object-Oriented Programming concepts covered in the OOP course to a small, self-contained C++ console game. Rather than practising encapsulation, composition, and constructors through disconnected textbook exercises, *Survival Days* requires several classes to cooperate within a single working system: a `Player` who holds resource state, a `Zombie` who threatens that state, and a `Game` that orchestrates the turn-based flow between them.

1.1 PROBLEM STATEMENT

Introductory and intermediate OOP assignments are frequently built around a single, isolated class such as a bank account or a shape, which demonstrates encapsulation but does not require multiple objects to interact, share state, or be composed into a larger system. As a result, it is common for students to be able to define OOP terms without having practised applying them together. At the same time, many student-built games are either too trivial to meaningfully exercise these concepts or too large in scope to complete within a single assignment window. This project is scoped to sit between those two extremes: small enough to finish within a course timeline, but structured around three interacting classes so that encapsulation, composition, and constructor-based initialization are all genuinely exercised.

1.2 PROJECT OBJECTIVES

To address the problem stated in Section 1.1, *Survival Days* is proposed with the following objectives:

1. **Design and implement cooperating classes:** Build a `Player`, `Zombie`, and `Game` class that interact with one another, clearly demonstrating encapsulation, composition, and constructor-based initialization.
2. **Model resource-management gameplay:** Balance health, hunger, food, and bullets turn by turn, requiring the player to plan and prioritize actions.
3. **Provide replayable, randomized gameplay:** Randomized search outcomes and randomized zombie encounter strength ensure no two playthroughs are identical.

4. **Reinforce programming and problem-solving skills:** Implementing the turn-based loop, input validation, and combat logic strengthens understanding of control flow, state management, and defensive programming.

1.3 SCOPE AND LIMITATIONS

Survival Days is developed as a single-player, text-based console application in C++ that can be compiled and run on any standard C++ compiler without additional dependencies.

The scope of this project is:

1. The target audience is any student or player with access to a C++ compiler and a terminal or console, requiring no specialized hardware or graphics support.
2. The system models core survival mechanics, health, hunger, combat, and looting, that can later be extended into a graphical or web-based version of the game.
3. Randomized event and combat systems give a different playthrough experience each time, providing meaningful replay value within a small codebase.

The limitations of this project are:

1. The game is entirely text- and console-based; no graphical user interface is provided in this version.
2. The game supports single-player mode only; there is no multiplayer or networked functionality.
3. There is no save/load system in the current version; progress is not retained between separate runs of the program.
4. The variety of zombie types and world content is intentionally limited to what is needed to clearly demonstrate OOP concepts, rather than to build a full commercial-scale game.

2. LITERATURE REVIEW

Games have been widely studied as a medium for teaching object-oriented concepts, on the basis that first-year programming students often disengage from purely textbook-driven exercises and respond better to interactive, goal-directed tasks. A recurring finding in this literature is that students understand OOP vocabulary, class, object, method, more readily when those terms map onto tangible, real-world-like entities that a game naturally provides, rather than abstract examples with no clear real-world analogue.

Text-based, turn-based survival simulations are a well-precedented format for this kind of exercise: a central game-management routine advances time in discrete steps (days or turns), while a player entity accumulates or loses resources based on the actions chosen and the random events encountered. This format is attractive for a course project because the core loop can be implemented with a small amount of code while still producing a genuinely replayable experience.

Design guidance on survival games more broadly identifies resource scarcity, typically modelled through health and hunger meters, as the mechanic that creates tension and rewards planning. This guidance also stresses that difficulty and death conditions should be tuned so failure feels like the result of the player's choices rather than an arbitrary punishment. These principles directly informed the tuning decisions in *Survival Days*, including the amount of hunger restored per meal and the randomized damage range assigned to each zombie, so that the game remains completable while still requiring the player to make meaningful trade-offs between searching, fighting, and eating.

3. METHODOLOGY

Given the small, single-developer scope of this project, development follows four sequential phases with light iteration between coding and testing as issues are found, rather than a multi-increment process.

Requirement Analysis: The game's functional requirements, the actions available to the player and the win/loss conditions, are identified and documented as a short specification describing each class's responsibilities.

Design: The system is designed against that specification. A UML class diagram, use case diagram, and sequence diagram (Section 4) are produced to model the classes, their relationships, and the order of interactions during gameplay.

Coding: The functionality specified during design is implemented in C++, producing a working console application.

Testing and Evaluation: Individual actions (search, fight, eat) are tested separately to confirm they update player state correctly, and the full game loop is tested end-to-end, including edge cases such as invalid menu input, running out of bullets mid-combat, and simultaneous health/hunger depletion, to confirm the game behaves correctly without crashing.

3.1 TOOLS TO BE USED

The following tools are used throughout this project to streamline the development process.

VS CODE:
Used as the primary editor for writing, navigating, and debugging the C++ source code.

GIT AND GITHUB:
Git is used to track changes and manage versions of the project throughout development. GitHub hosts the project repository and backs up the codebase.

G++ / GCC COMPILER:
The GNU C++ compiler (g++) is used to compile and build the console application, and to catch syntax and type errors during development.

3.2 OOP CONCEPTS COVERAGE

ENCAPSULATION:
`Player` and `Zombie` keep their attributes (health, hunger, bullets, food, damage) private, exposing controlled behaviour only through public methods such as `eat()`, `damage()`, `shoot()`, and `attack()`.

COMPOSITION:
The `Game` class owns a `Player` object for the lifetime of a session and creates `Zombie` objects on demand whenever a combat encounter occurs, modelling a clear "has-a" relationship between classes.

CONSTRUCTORS:
Each class defines a constructor that establishes its starting state: `Player` initializes health, hunger, bullets, and food to their starting values from the player's name; `Zombie` assigns a randomized damage value on creation; `Game` initializes the day counter and constructs its `Player`.

CONTROL FLOW AND STATE MANAGEMENT:
The `Game::start()` and `Game::fightZombie()` methods manage nested loops and conditional branching to drive the turn-based menu, the combat loop, and the end-of-game detection, reinforcing state-management and defensive-programming practice.

4. SYSTEM DESIGN

This section presents three design artifacts describing the system: the UML class diagram, which models the static structure of the classes and their relationships; the use case diagram, which models how the player interacts with the system's functionality; and the sequence diagram, which models the order of interactions during a combat encounter.

4.1 UML CLASS DIAGRAM

The class diagram below shows the three classes in the system, `Player`, `Zombie`, and `Game`, together with their attributes, methods, and relationships. The `Game` class is composed of exactly one `Player` object for the session, and creates `Zombie` objects as combat encounters occur, shown as a dependency relationship.

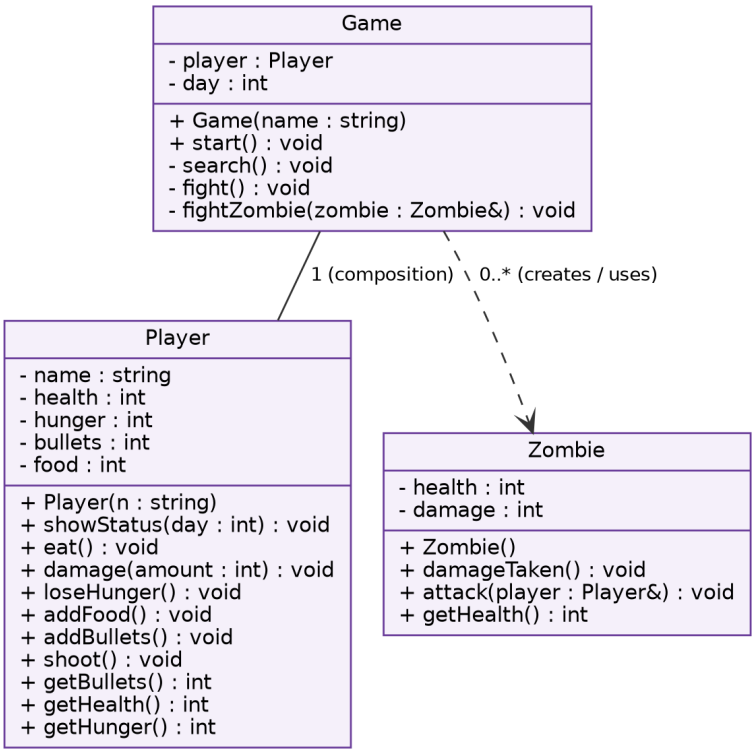

Figure 1: UML Class Diagram

4.2 USE CASE DIAGRAM

The use case diagram identifies the single actor, the Player, and the key actions available in a session: viewing the daily status, searching the surroundings, fighting a zombie, eating food, and quitting the game. Searching may include a random zombie encounter, which in turn includes the fight interaction, shown using the `<<include>>` relationship.

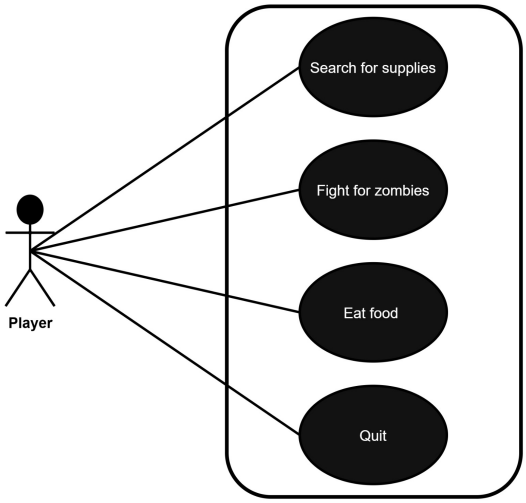

Figure 2: Use Case Diagram

4.3 SEQUENCE DIAGRAM

The sequence diagram illustrates the order of interactions among the User, `Game`, `Player`, and `Zombie` objects during a "Fight Zombie" encounter. It begins with the user choosing the fight option, for which `Game` creates a `Zombie` and checks the player's bullet count. It then shows the shoot/attack loop that repeats until either the player or the zombie is defeated, and the two outcomes that follow: the player receiving food for a defeated zombie, or the zombie attacking the player when it survives.

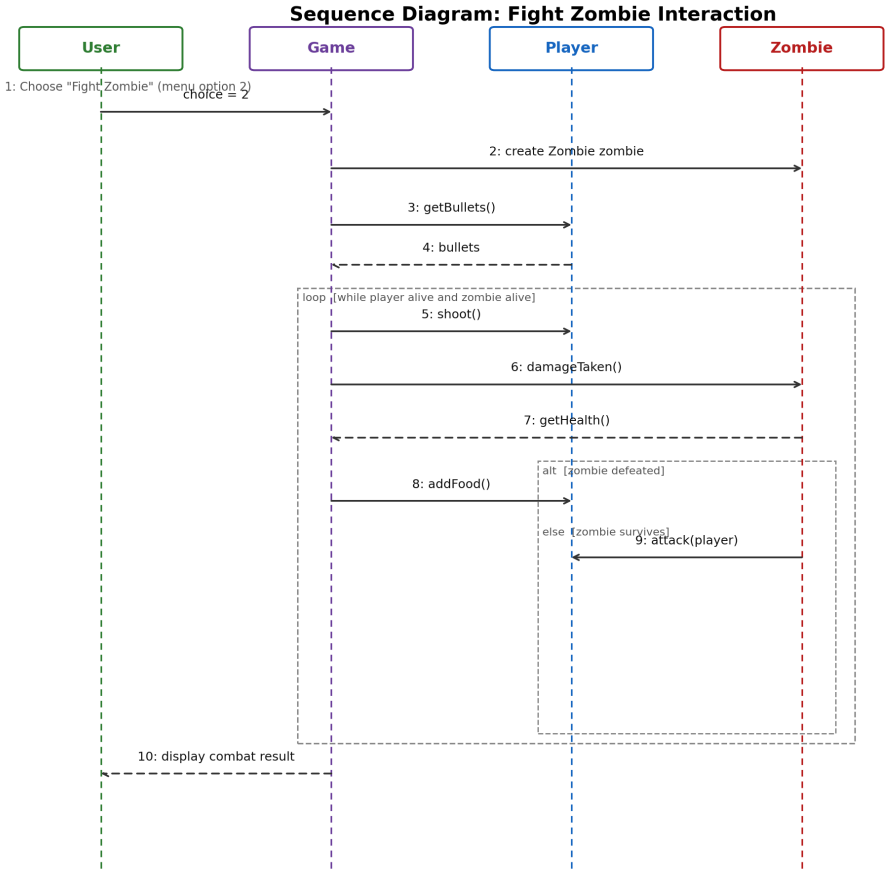

Figure 3: Sequence Diagram — Fight Zombie Interaction

5. PROPOSED DELIVERABLES

The project will deliver a single, self-contained C++ console application implementing a complete, playable turn-based survival game. There is one active user role: the Player, who chooses actions each day in an attempt to survive as long as possible.

In the finished application, the player will be able to search the surrounding area for supplies, fight zombies encountered directly or through a search-triggered ambush, eat food to restore hunger, and quit at any time to end the session early. The `Game` class manages the overall session: presenting the menu, applying passive hunger decay, resolving random search and combat outcomes, advancing the day counter, and detecting the game-over condition when health or hunger reaches zero.

The final deliverable also includes the well-commented C++ source code itself, together with the UML class, use case, and sequence diagrams presented in Section 4, so that the codebase and its design documentation together serve as a clear demonstration of the OOP concepts covered in this proposal.

BIBLIOGRAPHY

- Lippman, S. B., Lajoie, J., Moo, B. E. *C++ Primer*, 5th Edition. Addison-Wesley, 2012.
- C++ Language Reference, cppreference.com. `https://en.cppreference.com/`
- Git Documentation. `https://git-scm.com/doc`
- GitHub Docs. `https://docs.github.com/`
- Survival Game Design (Principles, Examples, Template), Game Design Skills. `https://gamedesignskills.com/game-design/survival/`